

Bharati Vidyapeeth

(Deemed to be University)

College of Engineering, Pune



LAB MANUAL

Subject: Mobile Application Development

B.Tech Information Technology

Semester -V

INDEX

Sr No.	Name of the Experiment
1	Create ―First Android Application‖ that will display ―BVDUCOEP-PUNE‖ in the middle of the screen in the green color with White background.
2	Develop an application that uses GUI components, Font and Colours.
3	Create login application where you will have to validate EmailID (UserName). Till the username and password is not validated, login button should remain disabled.
4	Developing of an application for data persistence.
5	Create an application that will change color of the screen, based on selected options from the menu.
6	Create an application that will display toast (Message) on specific interval of Time.
7	Create a background application that will open activity on specific Time.
8	Write an Android application for calculator.
9	Implement an application that creates an alert upon receiving a message.
10	Create simple app for Iso OS phone.

Experiment No 1 Objective:

To create a basic Android application that displays the text "BVDUCOEP-PUNE" in the middle of the screen with green color text on a white background.

Requirements

- Android Studio installed on your computer
- Basic knowledge of Java or Kotlin programming
- Basic understanding of XML for Android UI design

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "FirstAndroidApp".
    - Set the package name to `com.example.firstandroidapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Replace the existing `TextView` with a new one having the following attributes:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

  xmlns:tools="http://schemas.android.com/tools"

  android:layout_width="match_parent"

  android:layout_height="match_parent"

  android:background="#FFFFFF"

  tools:context=".MainActivity">
```

```
  <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="BVDUCOEP-PUNE"
    android:textColor="#008000"
    android:textSize="24sp"
    android:layout_centerInParent="true"/>
</RelativeLayout>
```

Step 3: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.firstandroidapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Ensure that your MainActivity class extends `AppCompatActivity` and sets the content view to the layout defined in `activity_main.xml`.

```
package com.example.firstandroidapp;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

  @Override

  protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

  }

}
```

Step 4: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 5: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, you should see the text "BVDUCOEP-PUNE" displayed in green color in the middle of the screen with a white background.

Conclusion

You have successfully created your first Android application that displays "BVDUCOEP-PUNE" in the middle of the screen with green text on a white background. This simple project helps you understand the basics of Android application development, including setting up a project, designing a layout, and writing the main activity code.

Experiment No 2 Objective:

To develop an Android application that incorporates various GUI components, custom fonts, and colors to create an interactive and visually appealing interface.

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "GUIComponentsApp".
    - Set the package name to `com.example.guicomponentsapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Add various GUI components such as `Button`, `EditText`, `TextView`, and customize their fonts and colors.

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:background="#FFFFFF"

    tools:context=".MainActivity">

    <TextView

        android:id="@+id/textView"

        android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
    android:text="Welcome to GUI Components App"
    android:textColor="#FF6200EE"
    android:textSize="24sp"
    android:fontFamily="@font/your_custom_font"
    android:layout_marginTop="20dp"
    android:layout_centerHorizontal="true"/>
<EditText
    android:id="@+id/editText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"
    android:layout_below="@id/textView"
    android:layout_marginTop="20dp"
    android:layout_marginLeft="20dp"
    android:layout_marginRight="20dp"/>
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:textColor="#FFFFFF"
    android:backgroundTint="#FF6200EE"
```

```
    android:layout_below="@id/editText"

    android:layout_marginTop="20dp"

    android:layout_centerHorizontal="true"/>

</RelativeLayout>
```

Step 3: Adding Custom Font

1. **Add Custom Font**:
    - Download a custom font (e.g., `custom_font.ttf`).
    - Place the font file in `res > font` (create the `font` directory if it doesn't exist).
2. **Reference the Font in XML**:
    - Add the `fontFamily` attribute to the `TextView` to use the custom font.

```
<TextView

   android:id="@+id/textView"

   android:layout_width="wrap_content"

   android:layout_height="wrap_content"

   android:text="Welcome to GUI Components App"

   android:textColor="#FF6200EE"

   android:textSize="24sp"

   android:fontFamily="@font/custom_font"

   android:layout_marginTop="20dp"

   android:layout_centerHorizontal="true"/>
```

Step 4: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.guicomponentsapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Implement the `Button` click functionality to display a greeting message.

```
package com.example.guicomponentsapp;

import android.os.Bundle;
```

```
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        EditText editText = findViewById(R.id.editText);
        Button button = findViewById(R.id.button);
        TextView textView = findViewById(R.id.textView);
        button.setOnClickListener(view -> {
            String name = editText.getText().toString();
            textView.setText("Hello, " + name + "!");
        });
    }
}
```

New File of code

```
package com.example.guicomponentsapp
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
```

```
import android.widget.TextView

import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        val editText = findViewById<EditText>(R.id.editText)

        val button = findViewById<Button>(R.id.button)

        val textView = findViewById<TextView>(R.id.textView)

        button.setOnClickListener {

            val name = editText.text.toString()

            textView.text = "Hello, $name!"

        }

    }

}
```

Step 5: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 6: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, you should see:
        - A welcome message in custom font and color.
        - An `EditText` to enter your name.

- A `Button` to submit the name.
- After entering your name and clicking the button, a greeting message should be displayed.

Conclusion

You have successfully developed an Android application that uses various GUI components, custom fonts, and colors. This project helps you understand the integration of different UI elements and customization techniques to enhance the user experience in Android development.

Experiment No 3 Objective :

To create an Android login application where the EmailID (Username) and password are validated. The login button remains disabled until the username and password are valid.

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "LoginApp".
    - Set the package name to `com.example.loginapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Create a layout with two `EditText` fields for the username and password, and a `Button` for login. The button should initially be disabled.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">
```

```
    <EditText
        android:id="@+id/editTextUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email ID"
        android:inputType="textEmailAddress"/>

    <EditText
        android:id="@+id/editTextPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"/>

    <Button
        android:id="@+id/buttonLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login"
        android:enabled="false"/>
</LinearLayout>
```

Step 3: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.loginapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Add logic to validate the EmailID and password, and enable the login button when both are valid.

```
package com.example.loginapp;

import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private EditText editTextUsername;
    private EditText editTextPassword;
    private Button buttonLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        editTextUsername = findViewById(R.id.editTextUsername);
        editTextPassword = findViewById(R.id.editTextPassword);
```

```
        buttonLogin = findViewById(R.id.buttonLogin);

        TextWatcher textWatcher = new TextWatcher() {
            @Override
            public void beforeTextChanged(CharSequence s, int start, int
count, int after) {}

            @Override
            public void onTextChanged(CharSequence s, int start, int before,
int count) {
                validateInputs();
            }

            @Override
            public void afterTextChanged(Editable s) {}
        };

        editTextUsername.addTextChangedListener(textWatcher);
        editTextPassword.addTextChangedListener(textWatcher);
    }

    private void validateInputs() {
        String username = editTextUsername.getText().toString().trim();
        String password = editTextPassword.getText().toString().trim();
        boolean isEmailValid =
android.util.Patterns.EMAIL_ADDRESS.matcher(username).matches();
        boolean isPasswordValid = !password.isEmpty();
        buttonLogin.setEnabled(isEmailValid && isPasswordValid);
    }
}
```

Step 4: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 5: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, try entering different values in the EmailID and password fields.
    - Ensure that the login button remains disabled until a valid EmailID and non-empty password are entered.

Conclusion

You have successfully created a login application that validates the EmailID (Username) and password. The login button remains disabled until both the username and password are valid. This project helps you understand input validation and dynamic UI updates in Android development.

Experiment No. 4 Objective:

To develop an Android application that demonstrates data persistence using SharedPreferences.

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "DataPersistenceApp".
    - Set the package name to `com.example.datapersistenceapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Replace the existing layout with the following code to add an EditText, a Button to save data, and a TextView to display saved data:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

  xmlns:tools="http://schemas.android.com/tools"

  android:layout_width="match_parent"

  android:layout_height="match_parent"

  tools:context=".MainActivity">
```

```
<EditText
   android:id="@+id/editTextData"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_centerHorizontal="true"
   android:layout_marginTop="50dp"
   android:hint="Enter some data"
   android:padding="10dp"
   android:background="#E0E0E0"/>

<Button
   android:id="@+id/buttonSave"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_below="@id/editTextData"
   android:layout_centerHorizontal="true"
   android:layout_marginTop="20dp"
   android:text="Save Data"/>

<TextView
   android:id="@+id/textViewDisplay"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:layout_below="@id/buttonSave"
   android:layout_centerHorizontal="true"
   android:layout_marginTop="20dp"
```

```
    android:textSize="18sp"/>

</RelativeLayout>
```

Step 3: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.datapersistenceapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Implement the functionality to save data to SharedPreferences and retrieve it when the app starts.

```
package com.example.datapersistenceapp;


import android.content.SharedPreferences;

import android.os.Bundle;

import android.view.View;

import android.widget.Button;

import android.widget.EditText;

import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {

  private EditText editTextData;

  private Button buttonSave;

  private TextView textViewDisplay;

  private SharedPreferences sharedPreferences;


  @Override

  protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        editTextData = findViewById(R.id.editTextData);
        buttonSave = findViewById(R.id.buttonSave);
        textViewDisplay = findViewById(R.id.textViewDisplay);

        sharedPreferences = getSharedPreferences("MyPrefs", MODE_PRIVATE);

        String savedData = sharedPreferences.getString("dataKey", "No Data Saved");
        textViewDisplay.setText(savedData);

        buttonSave.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String data = editTextData.getText().toString();
                SharedPreferences.Editor editor = sharedPreferences.edit();
                editor.putString("dataKey", data);
                editor.apply();
                textViewDisplay.setText(data);
            }
        });
    }
}
```

Step 4: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 5: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, enter some text in the EditText field and click the "Save Data" button.
    - The entered text should be displayed below the button.
    - Close and reopen the application to ensure that the data is persisted and displayed.

Conclusion

You have successfully developed an Android application that demonstrates data persistence using SharedPreferences. This project helps you understand how to save and retrieve data locally within an Android application.

Experiment No. 5 Objective:

To create an Android application that changes the background color of the screen based on the selected option from the menu.

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "ColorChangerApp".
    - Set the package name to `com.example.colorchangerapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.

2. **Modify the Layout**:
    - Ensure it has a basic layout, as the color change will happen in the entire layout.

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">

  <!-- This is the main layout whose background color will be changed -->
</RelativeLayout>
```

Step 3: Creating the Menu

1. **Create a Menu Resource File**:
    - Right-click on `res` folder, select `New > Android Resource File`.
    - Name the file `menu_main` and set the resource type to `Menu`. Click "OK".
2. **Define Menu Items**:
    - Open `menu_main.xml` and add the following items:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <item
    android:id="@+id/red"
    android:title="Red"/>
  <item
    android:id="@+id/green"
    android:title="Green"/>
  <item
    android:id="@+id/blue"
```

```
    android:title="Blue"/>
</menu>
```

Step 4: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.colorchangerapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Inflate the menu and handle item selections to change the background color.

```
package com.example.colorchangerapp;


import android.os.Bundle;

import android.view.Menu;

import android.view.MenuInflater;

import android.view.MenuItem;

import android.widget.RelativeLayout;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


  private RelativeLayout layout;


  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);


    layout = findViewById(R.id.main_layout);
  }
```

```
  @Override
  public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menu_main, menu);
    return true;
  }

  @Override
  public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
      case R.id.red:

layout.setBackgroundColor(getResources().getColor(android.R.color.holo_red_light));
        return true;
      case R.id.green:

layout.setBackgroundColor(getResources().getColor(android.R.color.holo_green_light));
        return true;
      case R.id.blue:

layout.setBackgroundColor(getResources().getColor(android.R.color.holo_blue_light));
        return true;
      default:
        return super.onOptionsItemSelected(item);
    }
  }
```

}

Step 5: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 6: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, click on the menu options.
    - Selecting "Red", "Green", or "Blue" should change the background color of the screen accordingly.

Conclusion

You have successfully created an Android application that changes the screen's background color based on the selected option from the menu. This project helps you understand the basics of menu creation and handling user interactions in Android.

Experimental No. 6 Objective :

To create an Android application that displays toast messages at specific intervals of time.

Requirements

- Android Studio installed on your computer
- Basic knowledge of Java or Kotlin programming
- Basic understanding of XML for Android UI design

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".

- Name your application "ToastIntervalApp".
- Set the package name to `com.example.toastintervalapp`.
- Set the save location as per your preference.
- Choose the language (Java or Kotlin).
- Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
- Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
   - Replace the existing layout with the following code to keep it simple:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Toast Interval App"
        android:textSize="24sp"
        android:layout_centerInParent="true"/>
</RelativeLayout>
```

Step 3: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.toastintervalapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
   - Write the code to display a toast message at specific intervals using a `Handler` or `Timer`.

Java:

```
package com.example.toastintervalapp;

import android.os.Bundle;
import android.os.Handler;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private Handler handler = new Handler();
```

```
    private Runnable runnable;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        runnable = new Runnable() {
            @Override
            public void run() {
                Toast.makeText(MainActivity.this, "This is a Toast message",
Toast.LENGTH_SHORT).show();
                handler.postDelayed(this, 5000); // 5 seconds interval
            }
        };

        handler.postDelayed(runnable, 5000); // Initial delay of 5 seconds
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        handler.removeCallbacks(runnable); // Remove callbacks to avoid
memory leaks
    }
}
```

Step 4: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 5: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, you should see a toast message "This is a Toast message" displayed every 5 seconds.

Conclusion

You have successfully created an Android application that displays toast messages at specific intervals. This project helps you understand how to use handlers or timers to execute code repeatedly in Android.

Experiment No. 07 Objective:

To create an Android application that runs in the background and opens a specified activity at a specific time using `AlarmManager`.

Requirements

- Android Studio installed on your computer
- Basic knowledge of Java or Kotlin programming
- Basic understanding of Android components such as Activities, BroadcastReceivers, and AlarmManager

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "TimedActivityOpener".
    - Set the package name to `com.example.timedactivityopener`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Add a button to set the alarm:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

  xmlns:tools="http://schemas.android.com/tools"

  android:layout_width="match_parent"

  android:layout_height="match_parent"

  tools:context=".MainActivity">
```

```
    <Button
        android:id="@+id/btnSetAlarm"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Set Alarm"
        android:layout_centerInParent="true"/>
</RelativeLayout>
```

Step 3: Creating the AlarmReceiver

1. **Create a new Java/Kotlin class**: Right-click on the `com.example.timedactivityopener` package and select "New > Java Class" or "New > Kotlin Class/File". Name it `AlarmReceiver`.
2. **Extend `BroadcastReceiver`**:

```
package com.example.timedactivityopener;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;

public class AlarmReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Intent activityIntent = new Intent(context, TargetActivity.class);
        activityIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        context.startActivity(activityIntent);
    }
```

}

Step 4: Creating the Target Activity

1. **Create a new Activity**: Right-click on the `com.example.timedactivityopener` package and select "New > Activity > Empty Activity". Name it `TargetActivity`.
2. **Modify the Layout**:
    - Open `activity_target.xml` and set a simple layout, such as a `TextView`:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".TargetActivity">

  <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Alarm Triggered!"
    android:layout_centerInParent="true"/>
</RelativeLayout>
```

Step 5: Modifying the MainActivity

1. **Open the `MainActivity` file**: Navigate to `java > com.example.timedactivityopener > MainActivity.java` (or `.kt` for Kotlin).
2. **Set the Alarm using `AlarmManager`**:

```
package com.example.timedactivityopener;

import android.app.AlarmManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;

import java.util.Calendar;

public class MainActivity extends AppCompatActivity {
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    Button btnSetAlarm = findViewById(R.id.btnSetAlarm);
    btnSetAlarm.setOnClickListener(v -> setAlarm());
  }

  private void setAlarm() {
    AlarmManager alarmManager = (AlarmManager) getSystemService(ALARM_SERVICE);
    Intent intent = new Intent(this, AlarmReceiver.class);
    PendingIntent pendingIntent = PendingIntent.getBroadcast(this, 0, intent, 0);
```

```
    Calendar calendar = Calendar.getInstance();
    calendar.setTimeInMillis(System.currentTimeMillis());
    calendar.set(Calendar.HOUR_OF_DAY, 14); // Set the hour for the alarm
    calendar.set(Calendar.MINUTE, 30); // Set the minute for the alarm

    alarmManager.setExact(AlarmManager.RTC_WAKEUP, calendar.getTimeInMillis(),
pendingIntent);
  }
}
```

Step 6: Adding Permissions to `AndroidManifest.xml`

1. **Open `AndroidManifest.xml`**: Navigate to `app > manifests > AndroidManifest.xml`.
2. **Add the Receiver and Permissions**:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.timedactivityopener">

  <application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.TimedActivityOpener">
    <activity android:name=".TargetActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
```

```
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".MainActivity" />

    <receiver android:name=".AlarmReceiver" />
  </application>
</manifest>
```

Step 7: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 8: Verify the Output

1. **Set the Alarm**:
    - Launch the application on your device or emulator.
    - Tap the "Set Alarm" button.
2. **Check if the Target Activity Opens**:
    - At the specified time, the TargetActivity should open, displaying the text "Alarm Triggered!".

Conclusion

You have successfully created an Android application that runs in the background and opens a specified activity at a specific time using `AlarmManager`. This project helps you understand the basics of using background services and scheduling tasks in Android.

Expremental No. 08 Objective:

To create a basic Android calculator application that can perform simple arithmetic operations such as addition, subtraction, multiplication, and division.

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
    - Click on "Start a new Android Studio project".
    - Choose "Empty Activity" and click "Next".
    - Name your application "CalculatorApp".
    - Set the package name to `com.example.calculatorapp`.
    - Set the save location as per your preference.
    - Choose the language (Java or Kotlin).
    - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
    - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Replace the existing layout with the following XML code to create a simple calculator interface:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity">



    <EditText

        android:id="@+id/editText"

        android:layout_width="match_parent"

        android:layout_height="wrap_content"

        android:layout_alignParentTop="true"
```

```
    android:layout_margin="16dp"
    android:inputType="none"
    android:textSize="24sp"/>

<GridLayout
    android:id="@+id/gridLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_below="@id/editText"
    android:layout_margin="16dp"
    android:columnCount="4"
    android:rowCount="5">

    <Button
        android:id="@+id/button0"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:text="0" />

    <Button
        android:id="@+id/button1"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:layout_columnWeight="1"
```

```
    android:layout_rowWeight="1"
    android:text="1" />

<Button
    android:id="@+id/button2"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="2" />

<Button
    android:id="@+id/button3"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="3" />

<Button
    android:id="@+id/button4"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="4" />
```

```
<Button
    android:id="@+id/button5"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="5" />

<Button
    android:id="@+id/button6"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="6" />

<Button
    android:id="@+id/button7"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="7" />

<Button
```

```
    android:id="@+id/button8"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="8" />

<Button
    android:id="@+id/button9"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="9" />

<Button
    android:id="@+id/buttonAdd"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="+" />

<Button
    android:id="@+id/buttonSub"
    android:layout_width="0dp"
```

```
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="-" />

<Button
    android:id="@+id/buttonMul"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="*" />

<Button
    android:id="@+id/buttonDiv"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
    android:layout_rowWeight="1"
    android:text="/" />

<Button
    android:id="@+id/buttonClear"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_columnWeight="1"
```

```
            android:layout_rowWeight="1"

            android:text="C" />


        <Button

            android:id="@+id/buttonEqual"

            android:layout_width="0dp"

            android:layout_height="0dp"

            android:layout_columnWeight="1"

            android:layout_rowWeight="1"

            android:text="=" />

    </GridLayout>

</RelativeLayout>
```

Step 3: Writing the MainActivity Code

1. **Open the `MainActivity` file**: Navigate to `java > com.example.calculatorapp > MainActivity.java` (or `.kt` for Kotlin).
2. **Modify the MainActivity Code**:
    - Implement the logic for handling button clicks and performing arithmetic operations.

```
package com.example.calculatorapp;


import android.os.Bundle;

import android.view.View;

import android.widget.Button;

import android.widget.EditText;

import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
```

```
private EditText editText;
private double valueOne = Double.NaN;
private double valueTwo;
private char currentAction;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    editText = findViewById(R.id.editText);
    Button button0 = findViewById(R.id.button0);
    Button button1 = findViewById(R.id.button1);
    Button button2 = findViewById(R.id.button2);
    Button button3 = findViewById(R.id.button3);
    Button button4 = findViewById(R.id.button4);
    Button button5 = findViewById(R.id.button5);
    Button button6 = findViewById(R.id.button6);
    Button button7 = findViewById(R.id.button7);
    Button button8 = findViewById(R.id.button8);
    Button button9 = findViewById(R.id.button9);
    Button buttonAdd = findViewById(R.id.buttonAdd);
    Button buttonSub = findViewById(R.id.buttonSub);
    Button buttonMul = findViewById(R.id.buttonMul);
    Button buttonDiv = findViewById(R.id.buttonDiv);
    Button buttonClear = findViewById(R.id.buttonClear);
    Button buttonEqual = findViewById(R.id.buttonEqual);
```

```
button0.setOnClickListener(v -> editText.append("0"));
button1.setOnClickListener(v -> editText.append("1"));
button2.setOnClickListener(v -> editText.append("2"));
button3.setOnClickListener(v -> editText.append("3"));
button4.setOnClickListener(v -> editText.append("4"));
button5.setOnClickListener(v -> editText.append("5"));
button6.setOnClickListener(v -> editText.append("6"));
button7.setOnClickListener(v -> editText.append("7"));
button8.setOnClickListener(v -> editText.append("8"));
button9.setOnClickListener(v -> editText.append("9"));

buttonAdd.setOnClickListener(v -> {
    computeCalculation();
    currentAction = '+';
    editText.setText(null);
});

buttonSub.setOnClickListener(v -> {
    computeCalculation();
    currentAction = '-';
    editText.setText(null);
});

buttonMul.setOnClickListener(v -> {
    computeCalculation();
    currentAction = '*';
```

```
        editText.setText(null);
    });

    buttonDiv.setOnClickListener(v -> {
        computeCalculation();
        currentAction = '/';
        editText.setText(null);
    });

    buttonClear.setOnClickListener(v -> {
        if (editText.getText().length() > 0) {
            CharSequence currentText = editText.getText();
            editText.setText(currentText.subSequence(0, currentText.length() - 1));
        } else {
            valueOne = Double.NaN;
            valueTwo = Double.NaN;
            editText.setText("");
        }
    });

    buttonEqual.setOnClickListener(v -> {
        computeCalculation();
        editText.setText(String.valueOf(valueOne));
        valueOne = Double.NaN;
        currentAction = '0';
    });
}
```

```
    private void computeCalculation() {
        if (!Double.isNaN(valueOne)) {
            valueTwo = Double.parseDouble(editText.getText().toString());
            switch (currentAction) {
                case '+':
                    valueOne = valueOne + valueTwo;
                    break;
                case '-':
                    valueOne = valueOne - valueTwo;
                    break;
                case '*':
                    valueOne = valueOne * valueTwo;
                    break;
                case '/':
                    valueOne = valueOne / valueTwo;
                    break;
            }
        } else {
            valueOne = Double.parseDouble(editText.getText().toString());
        }
    }
}
```

Step 4: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.

- Select your device or emulator from the list.
- Click "OK" to deploy the application.

Step 5: Verify the Output

1. **Check the Output**:
   - Once the application is installed and launched on your device or emulator, you should see a functional calculator interface.
   - Test all buttons and ensure that the arithmetic operations work correctly.

Conclusion

You have successfully created an Android calculator application that performs basic arithmetic operations. This project helps you understand the integration of UI components and basic logic implementation in Android development.

Objective

To create an Android application that displays an alert dialog when a message is received.

Requirements

- Android Studio installed on your computer
- Basic knowledge of Java or Kotlin programming
- Basic understanding of XML for Android UI design

Steps

Step 1: Setting Up the Project

1. **Open Android Studio**: Launch Android Studio on your computer.
2. **Start a New Android Studio Project**:
   - Click on "Start a new Android Studio project".
   - Choose "Empty Activity" and click "Next".
   - Name your application "AlertMessageApp".
   - Set the package name to `com.example.alertmessageapp`.
   - Set the save location as per your preference.
   - Choose the language (Java or Kotlin).
   - Set the Minimum API level to 21 (Android 5.0 Lollipop) or higher.
   - Click "Finish" to create the project.

Step 2: Designing the Layout

1. **Open the `activity_main.xml` file**: Navigate to `res > layout > activity_main.xml`.
2. **Modify the Layout**:
    - Design a simple layout that includes a `TextView` to display a placeholder message:

```
<?xml version="1.0" encoding="utf-8"?>

<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    tools:context=".MainActivity">


    <TextView

        android:id="@+id/textView"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Waiting for message..."

        android:textSize="18sp"

        android:layout_centerInParent="true"/>

</RelativeLayout>
```

Step 3: Creating the BroadcastReceiver

1. **Create a New Java/Kotlin Class**:
    - Right-click on the package name (`com.example.alertmessageapp`), select `New > Java Class` (or `New > Kotlin Class`), and name it `MessageReceiver`.
2. **Define the BroadcastReceiver**:
    - Add code to handle the broadcasted message and show an alert dialog:

```
package com.example.alertmessageapp;


import android.content.BroadcastReceiver;
```

```
import android.content.Context;
import android.content.Intent;
import androidx.appcompat.app.AlertDialog;

public class MessageReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        // Extract the message from the Intent
        String message = intent.getStringExtra("message");

        // Show an alert dialog with the message
        AlertDialog.Builder builder = new AlertDialog.Builder(context);
        builder.setTitle("New Message")
                .setMessage(message)
                .setPositiveButton("OK", null)
                .show();
    }
}
```

Step 4: Registering the BroadcastReceiver

1. **Open the `AndroidManifest.xml` file**: Navigate to `app > manifests > AndroidManifest.xml`.
2. **Add the BroadcastReceiver**:
    - Register the `MessageReceiver` in the manifest file:

```
<application
    ... >
    <receiver android:name=".MessageReceiver">
        <intent-filter>
```

```
        <action android:name="com.example.alertmessageapp.SHOW_ALERT"/>
    </intent-filter>
  </receiver>
</application>
```

Step 5: Sending a Broadcast Message

1. **Modify the `MainActivity` Code**:
    - Add a method to send a broadcast message and update the UI to include a button for testing:

```
package com.example.alertmessageapp;


import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);


    Button sendMessageButton = findViewById(R.id.sendMessageButton);
    sendMessageButton.setOnClickListener(new View.OnClickListener() {
      @Override
      public void onClick(View v) {
        Intent intent = new Intent("com.example.alertmessageapp.SHOW_ALERT");
```

```
            intent.putExtra("message", "Hello, this is a test message!");
            sendBroadcast(intent);
        }
    });
  }
}
```

Update the `activity_main.xml` Layout:

- Add a button to send the broadcast message:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:context=".MainActivity">

  <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Waiting for message..."
    android:textSize="18sp"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="50dp"/>

  <Button
```

android:id="@+id/sendMessageButton"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

android:text="Send Message"

android:layout_below="@id/textView"

android:layout_centerHorizontal="true"

Step 6: Running the Application

1. **Connect an Android Device or Start an Emulator**: Connect your Android device via USB or start an Android emulator.
2. **Run the Application**:
    - Click on the "Run" button (green play button) in Android Studio.
    - Select your device or emulator from the list.
    - Click "OK" to deploy the application.

Step 7: Verify the Output

1. **Check the Output**:
    - Once the application is installed and launched on your device or emulator, press the "Send Message" button.
    - An alert dialog should appear displaying the message "Hello, this is a test message!".

Conclusion

You have successfully created an Android application that displays an alert dialog when a message is received. This project helps you understand the basics of working with `BroadcastReceiver`, handling broadcasts, and creating alert dialogs in Android.